

Assignment 2 - CIS*4780 - Computational Intelligence

Deadline: February 23, 2026, (11:59pm)

Grade: 25%

Note 1: This is a single-person assignment. You must complete it on your own.

Note 2: The assignment must be implemented in PyTorch or Keras.

Note 3: The TAs will run the Jupyter notebooks. Make sure to save the results and also ensure that the code is working. You should submit **one** Jupyter notebook file.

- Please use Jupyter notebooks (**NOT** Jupyterlabs) for consistency.

Note 4: You should write clean code in a consistent format and include comments.

Note 6: Submit your assignment in HTML and .ipynb via Courselink, in the Dropbox tab. Do not submit zip or PDF files.

Note 7: The grading for this assignment will differ from that of Assignment 1. You will be required to meet with the TAs to run your program, explain the various sections of your assignment on the TA's computer, and discuss the process you followed to complete each section.

Note 8: It is expected that you complete the assignment on your own. Any form of copying, using existing code from the web, plagiarizing, or having someone else write your code is considered cheating.

Note 9: Follow the recommendations included at the end of this document as you see fit.

Description: The goal of this assignment is to label images by implementing different Neural Network classifiers. You will implement functions that are needed to initialize, train, evaluate, and make predictions. You also need to analyze the results and explain your observations.

This assignment may require the use of the Google Colab environment if your computer does not have sufficient computational capacity. As a student, you can obtain free access to Google Colab platform (GPU or TPU).

Important:

- **Emphasis on Analysis:** Significant focus will be placed on evaluating your results, including plotting diagrams and tables to enhance clarity.
- **Parameter Stability:** In Part 1.1, maintain consistent parameter settings to ensure reliable comparisons.

Step 1: Download, preprocess, and prepare training and testing data

For this assignment, you will use the CIFAR-10 Dataset. You can download the data from <https://www.cs.toronto.edu/~kriz/cifar.html>. This website provides a detailed description of images and their label. Use the Python version for this assignment.

This data **needs preprocessing**. You can use

- The CIFAR10 guide which is attached to this assignment
OR
- The following link (image processing section) to prepare your data:
<https://medium.com/@rhythm10/image-preprocessing-for-cifar-10-dataset-f2b5cdb221bb>

Load all data batches for training and testing into your working environment.

- Ensure that you only train the model on the training batches and test it on the testing batch. **Do not** use the testing set in the training process or for optimizing hyperparameters.
- Use training batches to create training and validation sets to build the classifier and optimize hyperparameters.

Use K-fold crossvalidation for **Parts 2.2, and 3.1** as follows:

- You can use implement 5-fold crossvalidation and use the 5 batches provided for training (4 batches) and validation (1 batch).
OR
- You can combine all 5 batches and allow a k-fold crossvalidation function to partition the data. This way, you can use other values of **k**. But, note that large **k** values may mean longer training.

Note: If some sections of the assignment are taking too long to execute (several hours), you can reduce the number of training batches. For example, consider using 2 or 3 batches instead of 4.

Step 2: Follow these instructions to build the algorithms:

- **Part 1:** For this part, you will implement feedforward neural network models with the **Sigmoid activation** function for the hidden layers, and **Softmax** on the output layer. You must use **cross-entropy** loss as the optimization function and **mini-batch gradient descent** (MGD) for training. Note that the Softmax function is combined with Cross-entropy loss in some libraries.
 - i. You must initialize the weights and biases in the network. For this part of the assignment, you must use the following initialization methods:
 - 1. Random initialization: The weights are initialized randomly to a small value from a uniform distribution. The bias parameters are initialized to zero.
 - 2. Weights are initialized to 0.
 - ii. You should report: **1) the error rate for the training and testing processes** for the following tasks and **2) the analysis of using different values** for the following parameters:
 - a. Set the batch size to 250.
 - b. Try different values of **epochs**.
 - i. Plot error rate vs epochs. Explain the results.
 - c. **Use one** hidden layer and different numbers of hidden nodes, **between 2 to 300 nodes**.
 - i. Explain the results (plot diagrams or include tables if needed)
 - d. Implement the two initialization methods (weight and bias parameters) specified above.
 - i. Compare and explain the results.
 - e. Try different learning rate between **[0.0005 0.5]**
 - i. Explain the results (plot diagrams or include tables if needed)
 - f. Try different numbers of hidden layers, between **1 - 8**
 - i. Explain the results (plot diagrams or include tables if needed)
 - g. Implement a dropout method for regularization, rate between **0.2-0.5**, and report the results and your observation
 - i. Explain the results (plot diagrams or include tables if needed)

- **Part 2:** For this part, you will implement deep neural network models with multiple hidden layers, the **ReLU activation** function for the hidden layers and **Softmax** on the output layer. You must use **cross-entropy**. Note that the Softmax function is combined with Cross-entropy loss in some libraries.

1) **Part 2.1:** You should report: **1) the error rate and f-measure for training and testing set** and **2) your observations**.

- Set the parameters as follows and report the results:
 - Use **mini-batch gradient descent** (MGD) for training
 - Use random initialization for weight and bias parameters;
 - Select the best number of hidden units from Part 1.1;
 - Select the best value for the hidden layers from Part 1.1;
 - Select the best value for learning rate from Part 1.1;
 - Change different values of epochs; select the best value.

For analysis, plot diagrams or include tables if needed to better support your reasoning.

Note, for Part 2.1, you **do not need** to use k-fold crossvalidation.

- Part 2.2:** Use a train-and-validation (k-fold crossvalidation) function to optimize the parameters of the model.
 - You should report the average: **1) Error** on training, validation, and testing sets and **2) F-measure** on training, validation, and testing sets.
 - Use **batch gradient descent** for training
 - Use an optimization method (grid search) to select the best parameters for the learning rate, epoch, and neurons.
 - Report these hyperparameters** resulting from this process, including execution time for training.
 - Compare** the use Sigmoid activation function (Part 1) and ReLU activation function in Part 2.2 (in terms of training time/efficiency or performance)
 - Compare** hyperparameters resulted in this activity with the parameters selected in Part 1.

- **Part 3: Comparative analysis** For this part, you are asked to compare Parts 2.1 and 2.2 results and explain your observation comparing the two classifiers. You may have to review reliable online sources (e.g. tutorials) to better understand the impact of various parameters and neural network designs on the classification performance.

- Comparison of Results:
 - Compare results from Parts 2.1 and 2.2:
 - Explain your observations comparing the classifiers.

- Investigate whether any model outperformed others and explain contributing factors.
 - Analyze error rates and F-measure alongside execution times of the models.
- **Part 4:** For the last part, implement and use the convolutional neural network (CNN) algorithm to classify CIFAR10 data.
 - **Note that input data points processing for this dataset will be different and input data must have a different format.**
 - For the fully connected layers
 - Use grid search for hyperparameter adjustment
 - Use RELU as the activation function (hidden layer)
 - Use Softmax/cross-entropy loss
 - Use batch size = 100 (or more)
 - Set your own convolutional layer parameters.
 - Report error and f-measure for CNN
 - Compare results for Part 2.2 and 4. Analyze results. Explain why one model has outperformed the other model.

Recommendations:

- For Part 1.1, **keep** parameters as stable as possible so that the comparison of different values of hidden nodes, epochs, weights, and learning rates will be reliable.
- You can **plot** diagrams and **create** tables to better explain results.
- **Do** research to find more effective ways to visualize and explain results.
- **Do** research to learn about different parameters and the impact of parameters on NN performance.
- **Make sure that you** focus on the models, their parameters and analytics aspects of the assignment. This is not a programming course and you are not marked for your programming skills.
- You will **not be marked** on the accuracy or error rates you obtain. But, your results should make sense and there should be a good justification for the results you are reporting.
- **Do not use** the convolutional neural network or any variation of that algorithm in Sections 1-3.
- **Do Not use** of generative AI in any components of this assignment.
- **Do explain** all figures and tables generated as part of the assignment. Students may lose mark if random figures and tables are included in their reports.